RateGuard

Testing guide: how to set up, run and test every feature,
step by step, with the result you should expect each time

Written for Windows and for someone working with Go for the first time. Every request in this guide was run against the real service before the guide was written, and the expected results are copied from those runs.

Tested against: v1 (all seven phases) plus the Postgres-outage fix of ADR 0021
Tools used: Docker Desktop, Postman, VS Code, PowerShell, Git Bash, a web browser
Date: 5 November 2026

Contents

Part 1. Setup

1.1 Which editor to use (and why not the others)

1.2 What to install

1.3 Start RateGuard

1.4 Set up Postman

1.5 How to read this guide

Part 2. Manual tests in Postman

T1–T2 Health checks

T3–T4 Admin security

T5 Register an application

T6 No policy means refused

T7 Create the default plan

T8 Burst until refused (429)

T9 Peek without spending

T10 Tokens come back over time

T11 Invalid API key

T12 Caller without a key

T13 Bad input is rejected

T14 Give the app its own plan

T15 A stricter limit on one endpoint

T16 Change a plan live

T17 Keys are never shown twice

T18 Key rotation

T19 Disabling a plan

T20 Admin input validation

T21 Unknown routes and wrong methods

T22 Request IDs

T23 Usage analytics

T24 Prometheus metrics

T25 Swagger UI (API docs)

Part 3. Failure tests

F1 Redis goes down

F2 Postgres goes down

F3 Secrets never reach the logs

Part 4. Automated tests

A1 Unit tests

A2 Unit tests with the race detector

A3 Token bucket coverage (90% floor)

A4 Lint

A5 Integration tests (real Postgres and Redis)

A6 Go middleware tests

A7 Three instances, one shared limit (demo)

A8 Database migrations down and up

A9 Load test (optional, 25 minutes)

Part 5. Finishing

5.1 Stop or reset everything

5.2 Troubleshooting

5.3 Checklist

What RateGuard is, in one paragraph

RateGuard is a small web service that answers one question for other applications: "*should I allow this request?*" An application sends RateGuard the method and path of a request it received (for example `GET /api/products`) plus the caller's API key. RateGuard finds the caller's plan ("5 requests per 5 minutes", say), takes one token from that caller's bucket, and answers **200 (allow)** or **429 (too many requests)**. Buckets live in Redis so several RateGuard copies share one limit. Plans, applications and keys live in PostgreSQL. You manage them through an admin API protected by an admin token.

Piece	What it is	Address on your PC
RateGuard	The Go service (in a Docker container)	`http://127.0.0.1:8080`
PostgreSQL	Database: plans, applications, keys, usage	`127.0.0.1:5433`
Redis	Fast shared memory: the token buckets	`127.0.0.1:6379`
Swagger UI	Web page documenting the API (optional)	`http://127.0.0.1:8081`

Part 1. Setup

1.1 Which editor to use

Tool you have	Use it for this project?	Why
VS Code	**Yes: your main editor**	Free, excellent Go support through one extension, and it has a built-in terminal for the Docker commands.
Postman	**Yes: for API testing**	Part 2 of this guide is done entirely in Postman.
IntelliJ IDEA	No	Go support needs the paid Ultimate edition with the Go plugin (or JetBrains' separate GoLand IDE). The free Community edition cannot run Go. VS Code does everything you need.
Android Studio	No	Built for Android apps; it has no Go support.

VS Code extensions to install

Open VS Code, click the **Extensions** icon on the left bar (four squares) or press `Ctrl+Shift+X`, search, and click **Install**:

Extension	Publisher	Needed?	What it gives you
Go	Go Team at Google	Required	Syntax colours, autocomplete, go-to-definition, a "run test" link above every test, formatting on save.
Docker (may be listed as "Container Tools")	Microsoft	Optional	See running containers and their logs in the side bar.
YAML	Red Hat	Optional	Nicer editing of `docker-compose.yml` and `api/openapi.yaml`.

After installing the **Go** extension, open any `.go` file. A pop-up in the bottom-right corner says some tools (such as `gopls`) are missing: click **Install All**. If you miss it, press `Ctrl+Shift+P`, type **Go: Install/Update Tools**, tick every box, and click **OK**. This needs Go installed first (next section).

1.2 What to install

Program	Needed for	How to check it is installed
Docker Desktop	Everything: RateGuard, Postgres and Redis all run in Docker	`docker version` shows a "Server" section
Git for Windows (includes **Git Bash**)	Getting the code; running the demo script (A7)	`git --version`
Go 1.27.3 (from go.dev/dl)	Part 4 unit tests and the VS Code Go extension. Not needed for Parts 2 and 3.	`go version` prints `go1.27.3`
Postman	Part 2	Already installed

You do **not** need to install PostgreSQL, Redis, `make` or `golangci-lint`: they all run inside Docker.

> **NOTE** The project's README uses `make` commands (`make up`, `make test`). `make` is not installed on Windows by default, so this guide gives the plain command each one runs instead.

Opening a terminal in VS Code

1. In VS Code: **File** → **Open Folder...** and choose `D:\learn\Side-Projects\RateGuard-Workspace\RateGuard`.
2. Open the terminal: menu **Terminal** → **New Terminal** (or `Ctrl+``). It opens **PowerShell** in the project folder.
3. When this guide says **Git Bash**, click the small ˅ arrow next to the **+** in the terminal panel and choose **Git Bash**.

CAREFUL In PowerShell, `curl` is not the real curl. This guide avoids it and uses Postman or the browser instead. If you ever do need curl in PowerShell, type `curl.exe`.

1.3 Start RateGuard

1. **Start Docker Desktop** from the Start menu. Wait until the whale icon in the taskbar stops animating and Docker Desktop says *Engine running* (under a minute).
2. In the VS Code terminal (PowerShell), make sure you are on the latest code:

   ```
   git checkout main
   git pull
   ```

3. Create your settings file if it does not exist yet (on your PC it already does; this command only copies when the file is missing, so it is always safe):

   ```
   if (-not (Test-Path .env)) { Copy-Item configs\.env.example .env }
   ```

   `.env` holds local settings, including the admin token `change-me-local-only-admin-token-0000` used throughout this guide.
4. **Start from an empty database.** The expected results in this guide assume nothing has been created yet. Your PC's database still holds test data from the load tests (including a default plan of 3 requests an hour), which would change the numbers you see. This command stops everything and **deletes all local RateGuard data** (only the data in Docker, never your code):

   ```
   docker compose down -v
   ```

5. Build and start everything (the first build takes a few minutes; later starts take seconds):

   ```
   docker compose up -d --build --wait
   ```

6. Check what is running:

   ```
   docker compose ps -a
   ```

EXPECTED The `up` command ends with lines like `Container rateguard-rateguard-1 Healthy`, `…-postgres-1 Healthy`, `…-redis-1 Healthy`. In `docker compose ps -a`, **rateguard**, **postgres** and **redis** are *Up (healthy)* and **migrate** is *Exited (0)*. That is correct: **migrate** is a one-off job that creates the database tables and then stops.

IF IT FAILS On a brand-new database, `up` occasionally stops with *dependency postgres failed to start*, because the first boot of Postgres restarts itself once. Just run the same `up` command again. If it says a port is *already allocated*, another program uses 8080, 5433 or 6379: change `RATEGUARD_HOST_PORT`, `POSTGRES_HOST_PORT` or `REDIS_HOST_PORT` in `.env` and run `up` again (and use the new port in Postman).

Quick check in the browser

Open `http://127.0.0.1:8080/readyz` in any browser. You should see `{"status":"ready","checks":{"postgres":"ok","redis":"ok"}}`. RateGuard is running.

1.4 Set up Postman

a) Create an environment (stores values you reuse)

1. In Postman's left side bar click **Environments**, then the **+** button. Name it **RateGuard Local**.
2. Add these variables (type the name in the **Variable** column and the value in the **Initial value** and **Current value** columns, or the single **Value** column in newer versions). Leave the last six empty: requests fill them in for you.

Variable	Value
`baseUrl`	`http://127.0.0.1:8080`
`adminToken`	`change-me-local-only-admin-token-0000`
`clientId`, `apiKey`, `keyId`, `apiKey2`, `freeId`, `premiumId`	*(leave empty)*

3. Click **Save** (or `Ctrl+S`).
4. In the top-right corner of Postman there is a drop-down that says **No environment**. Choose **RateGuard Local**. **Forgetting this is the most common mistake:** every `{{baseUrl}}` would then be blank.

b) Create a collection (a folder for your requests)

1. Click **Collections** in the left side bar, then **+**, and name it **RateGuard Tests**.
2. For each test: click the ⋯ next to the collection, choose **Add request**, give it the test's name, and fill it in as described. Saved requests can be re-sent later with one click.

c) The parts of a Postman request you will use

Where	What to do
Method drop-down (left of the URL bar)	Pick `GET`, `POST`, `PUT`, `PATCH` or `DELETE`.
URL bar	Type the URL exactly as shown, for example `{{baseUrl}}/v1/check`. Variables in `{{ }}` turn orange when Postman recognises them (red means the environment is not selected or the name is misspelt).
Authorization tab	**Auth Type** → **Bearer Token**, then type the token in the **Token** box, e.g. `{{adminToken}}`. For "no key" tests choose **No Auth**.
Body tab	Choose **raw**, then change the drop-down on the right from **Text** to **JSON**, and paste the JSON. (Postman then sends `Content-Type: application/json` for you.)
Headers tab	Only needed once (T22), to add your own header.
Scripts tab → **Post-response** (called **Tests** in older Postman)	Paste the small script shown in a test. It runs after the response arrives and saves values (like the new API key) into your environment.
Send button	Sends the request.

Reading the response (bottom half of the window)

- **Status**: top right of the response area, e.g. `200 OK` or `429 Too Many Requests`. In this service, the status code *is* the answer.
- **Body** tab with **Pretty** selected: the JSON reply.
- **Headers** tab: the response headers, for example `X-RateLimit-Remaining` and `X-Request-Id`.

1.5 How to read this guide

- Do the Part 2 tests **in order**. Later tests use values (keys, IDs, plans) that earlier tests create.
- Each test has a **request table** (what to fill in), an **Expected** box (what you should see) and a line on **what it proves**.
- IDs, keys and request IDs are random, so yours will differ from the examples. Things like `01a1abee-…` or `rg_sho_…` stand for those.
- Numbers that depend on time (`reset_after`, `retry_after`) count down while you work, so yours may be a few seconds lower.
- Every error response has the same shape: `{"error":"<message>","request_id":"<random id>"}`. The guide shows only the message.

TIP If a test goes wrong and you want to start over, run `docker compose down -v` then `docker compose up -d --build --wait`, clear the six empty variables in Postman, and restart at T1.

Part 2. Manual tests in Postman

Health, security, plans, limits, keys, analytics and metrics. Allow about 45 minutes.

T1. Liveness: is the process up?

Request	`GET` `{{baseUrl}}/healthz`
Authorization	No Auth

EXPECTED Status **200 OK**. Body `{"status":"ok"}`. In the response **Headers** tab there is an `X-Request-Id` with a random 32-character value.

Proves: the service is running. Every response carries a request ID for tracing.

T2. Readiness: can it reach its databases?

Request	`GET` `{{baseUrl}}/readyz`
Authorization	No Auth

EXPECTED Status **200 OK**. Body `{"status":"ready","checks":{"postgres":"ok","redis":"ok"}}`.

Proves: readiness really checks Postgres and Redis (Part 3 shows it turning red).

T3. Admin API refuses a missing token

Request	`GET` `{{baseUrl}}/v1/clients`
Authorization	No Auth

EXPECTED Status **401 Unauthorized**. Body error: `"a valid admin token is required"`. Headers include `Www-Authenticate: Bearer realm="rateguard-admin"`.

Proves: nobody can manage applications or plans without the admin token.

T4. Admin API refuses a wrong token

Request	`GET` `{{baseUrl}}/v1/clients`
Authorization	Bearer Token, Token: `wrong`

EXPECTED Status **401 Unauthorized**, same error as T3. Now change the token to `{{adminToken}}` and send again: **200 OK** with body `{"clients":[]}` (the database is empty).

Proves: only the exact admin token works.

T5. Register an application and get its API key

An "application" (called a *client* in the API) is a program that will call RateGuard. Registering it creates its first API key.

Request	`POST` `{{baseUrl}}/v1/clients`
Authorization	Bearer Token: `{{adminToken}}`
Body (raw, JSON)	`{"name": "Shop App"}`

Scripts → Post-response	`const r = pm.response.json();`
	`pm.environment.set("clientId", r.client.id);`
	`pm.environment.set("apiKey", r.api_key.key);`
	`pm.environment.set("keyId", r.api_key.id);`

EXPECTED Status **201 Created**. The body has two parts:

```
{"client":{"id":"01a1abee-…","name":"Shop App","policy_id":null,"status":"active",…},
 "api_key":{"id":"01a1abee-…","prefix":"rg_sho_d3tttpsmd3rk7gsz","active":true,"revoked_at":null,
            "key":"rg_sho_d3tttpsmd3rk7gsz_<52 more letters and digits: the secret part>"}}
```

The key starts with `rg_sho_` ("sho" from "Shop App"). Open the environment (eye icon, top right) and check that `clientId`, `apiKey` and `keyId` are now filled in.

Proves: registration works and the full key is returned. This is the **only** time RateGuard ever shows it: it stores just a SHA-256 fingerprint.

T6. With no plan at all, requests are refused

This is the main request of the whole service: "may this caller do `GET /api/products`?" You will send it many times. **Save it** as "Check".

Request	`POST` `{{baseUrl}}/v1/check`
Authorization	Bearer Token: `{{apiKey}}` (the application's key, **not** the admin token)
Body (raw, JSON)	`{"method": "GET", "path": "/api/products"}`

EXPECTED Status **403 Forbidden**. Error: `"no rate-limit policy applies to this caller"`.

Proves: a caller without a plan is never treated as "unlimited". RateGuard fails safe.

T7. Create the default plan "Free"

A *policy* is a plan: how many requests (`capacity`) per how many seconds (`window_seconds`). The **default** policy applies to anyone without a plan of their own.

Request	`POST` `{{baseUrl}}/v1/policies`
Authorization	Bearer Token: `{{adminToken}}`
Body (raw, JSON)	`{"name": "Free", "capacity": 5, "window_seconds": 300, "default": true}`
Scripts → Post-response	`pm.environment.set("freeId", pm.response.json().id);`

EXPECTED Status **201 Created**. Body:

```
{"id":"01a1abee-…","name":"Free","capacity":5,"window_seconds":300,
 "refill_rate":0.016666666666666666,"enabled":true,"default":true,…}
```

`refill_rate` is tokens per second: 5 ÷ 300, i.e. one token back every 60 seconds. The **Headers** tab shows `Location: /v1/policies/<id>`.

Proves: plans can be created, and the refill rate is worked out for you when you leave it out.

T8. Use up the bucket: five allowed, the sixth refused

Go back to the saved **Check** request from T6 and click **Send six times**, one after another (finish within about 50 seconds). After each click, look at the status, the body and the **Headers** tab.

Click	Status	Body
1	200 OK	`{"allowed":true,"limit":5,"remaining":4,"reset_after":60}`
2	200 OK	`…"remaining":3,"reset_after":118` (about)
3	200 OK	`…"remaining":2,"reset_after":176` (about)
4	200 OK	`…"remaining":1,"reset_after":234` (about)
5	200 OK	`…"remaining":0,"reset_after":292` (about)
6	**429 Too Many Requests**	`{"allowed":false,"limit":5,"remaining":0,"reset_after":290,"retry_after":50}` (about)

In the response **Headers** tab of every click: `X-RateLimit-Limit: 5`, `X-RateLimit-Remaining` equal to `remaining`, and `X-RateLimit-Reset` equal to `reset_after`. Click 6 also has `Retry-After: 50` (about), equal to `retry_after`.

EXPECTED Exactly five **200**s, then **429**. `remaining` goes 4, 3, 2, 1, 0. `reset_after` = seconds until the bucket is full again; `retry_after` = seconds until one token is back. Both are counts of seconds, not clock times. Your seconds will differ slightly depending on how fast you click.

Proves: the token bucket works and the 429 carries the standard rate-limit headers your app can copy onto its own response.

T9. Peek: ask without spending a token

Request	POST `{{baseUrl}}/v1/peek` (duplicate "Check": right-click → **Duplicate**, then change `check` to `peek`)
Authorization / Body	Same as T6

EXPECTED Status **200 OK** even though the bucket is empty. Body `{"allowed":false,"limit":5,"remaining":0,"reset_after":…,"retry_after":…}`. Send it again: still **200**, still `remaining: 0`.

Proves: peek reports what a check *would* do without taking a token. It always answers 200; read `allowed`.

T10. Tokens come back over time

Wait until the `retry_after` from T8 has passed (at most about 60 seconds), then send **Check** once.

EXPECTED **200 OK** with `"remaining":0`: one token had refilled and you just used it. Send again straight away: **429**.

Proves: the bucket refills steadily at the plan's rate (one token every 60 seconds for Free).

T11. An invalid API key is refused

Request	POST `{{baseUrl}}/v1/check`, body as T6
Authorization	Bearer Token: `rg_xxx_aaaaaaaaaaaaaaaa_bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb`

EXPECTED **401 Unauthorized**. Error: `"invalid API key"`. Try also a random word like `hello` as the token: same **401**.

Proves: a bad key is an error. It is never quietly treated as an anonymous caller.

T12. A caller with no key is limited by IP address

Request	POST `{{baseUrl}}/v1/check`, body as T6
Authorization	**No Auth**

EXPECTED **200 OK**, `{"allowed":true,"limit":5,"remaining":4,"reset_after":60}`. This caller has a fresh bucket of its own, separate from Shop App's empty one.

Proves: callers without keys use the default plan, counted per IP address.

T13. Bad input is rejected with a clear message

Use the **Check** request (Bearer `{{apiKey}}`) and change only the body each time:

Body to send	Expected status	Expected error
`{"method":"GET","path":"/api/products?page=2"}`	400	`path must not contain a query string or fragment; send the path only`
`{"path":"/api/products"}`	400	`method is required`
`{"method":`	400	`request body is not valid JSON`

Afterwards put the body back to `{"method": "GET", "path": "/api/products"}` and save.

Proves: input is validated. Query strings are refused so `/api/login?x=1` cannot sneak past a rule on `/api/login`.

T14. Give the application its own plan

Step 1: create a "Premium" plan (100 requests a minute).

Request	`POST` `{{baseUrl}}/v1/policies`, Bearer `{{adminToken}}`
Body	`{"name": "Premium", "capacity": 100, "window_seconds": 60}`
Post-response	`pm.environment.set("premiumId", pm.response.json().id);`

`EXPECTED` **201 Created**, `"capacity":100,"refill_rate":1.6666666666666667,"default":false`.

Step 2: put Shop App on Premium.

Request	`PUT` `{{baseUrl}}/v1/clients/{{clientId}}/policy`, Bearer `{{adminToken}}`
Body	`{"policy_id": "{{premiumId}}"}`

`EXPECTED` **200 OK**; the returned client now has `"policy_id"` equal to Premium's ID.

Step 3: send Check again.

`EXPECTED` **200 OK** with `"limit":100` straight away. `remaining` will **not** be 99: the bucket keeps the tokens it had (none) and now refills at Premium's faster speed (1.67 a second), so you see a number that grows the longer you waited (right after T10 it is `0`; a minute later it would be in the 90s).

Proves: each application can have its own plan, and a plan change applies at once without restarting anything.

T15. A stricter limit on one endpoint (login)

An *endpoint rule* gives one part of an API its own, separate limit inside a plan. Here: 3 login attempts per 5 minutes.

Step 1: add the rule to Premium.

Request	POST `{{baseUrl}}/v1/policies/{{premiumId}}/rules`, Bearer `{{adminToken}}`
Body	`{"method": "POST", "path_pattern": "/api/login", "capacity": 3, "window_seconds": 300}`

EXPECTED **201 Created**, `{"id":"…","policy_id":"…","method":"POST","path_pattern":"/api/login","capacity":3,"window_seconds":300,…}`. GET `{{baseUrl}}/v1/policies/{{premiumId}}/rules` lists it under `"rules"`.

Step 2: duplicate **Check**, name it "Check login", set the body to `{"method": "POST", "path": "/api/login"}`, and send it **four** times.

EXPECTED **200** `"limit":3,"remaining":2` → **200** `"remaining":1` → **200** `"remaining":0` → **429** `{"allowed":false,"limit":3,"remaining":0,…,"retry_after":…}`.

Step 3: try to dodge the rule. Change the path to `/api/login/` (extra slash) and send.

EXPECTED Still **429**, `"limit":3`. Paths are cleaned before matching.

Step 4: other traffic is unaffected. Change the method to `"GET"` (path `/api/login`) and send; then send the normal **Check**.

EXPECTED Both **200 OK** with `"limit":100`: the rule only covers `POST /api/login`, and its bucket is separate from the plan's main bucket.

Proves: endpoint rules apply, can't be dodged with path tricks, and don't use up the main allowance.

T16. Change a plan while it is in use

Request	PATCH `{{baseUrl}}/v1/policies/{{premiumId}}`, Bearer `{{adminToken}}`
Body	`{"capacity": 200}`

EXPECTED **200 OK**, `"capacity":200,"refill_rate":3.3333333333333335` (recomputed for you). Send **Check** right away: **200** with `"limit":200`.

Proves: changes reach the hot path immediately. Each copy of RateGuard caches plans, and every change is broadcast over Redis to clear those caches.

T17. Listing data never shows a key again

With Bearer `{{adminToken}}`, send each of these:

Request	Expected (200 OK)
GET `{{baseUrl}}/v1/clients`	`{"clients":[{"id":"…","name":"Shop App","policy_id":"<Premium id>",…}]}`
GET `{{baseUrl}}/v1/clients/{{clientId}}`	The one client
GET `{{baseUrl}}/v1/clients/{{clientId}}/keys`	`{"api_keys":[{"id":"…","prefix":"rg_sho_…","active":true,"revoked_at":null,…}]}`
GET `{{baseUrl}}/v1/policies`	Both plans, Free with `"default":true`

EXPECTED The keys list shows only the `prefix`. There is **no** `"key"` field anywhere.

Proves: a key can't be read back after creation, even by an admin.

T18. Key rotation without downtime

Real systems change keys from time to time. RateGuard lets an application hold two keys at once: issue new → switch callers → revoke old.

#	Do this	Expected
1	POST `{{baseUrl}}/v1/clients/{{clientId}}/keys`, Bearer `{{adminToken}}`, no body. Post-response: `pm.environment.set("apiKey2", pm.response.json().key);`	**201 Created** with a new `"key":"rg_sho_…"` (different from the first)
2	Send the same request again	**409 Conflict**: `client already has 2 active API keys; revoke one before issuing another`
3	**Check** with Bearer `{{apiKey2}}`	**200**, `"limit":200`: both keys share one bucket
4	DELETE `{{baseUrl}}/v1/clients/{{clientId}}/keys/{{keyId}}`, Bearer `{{adminToken}}`	**204 No Content** (empty body)
5	**Check** with the old `{{apiKey}}`	**401**: `invalid API key`
6	**Check** with `{{apiKey2}}`	**200**

Proves: rotation works, a third key is blocked, and a revoked key stops working immediately.

FROM NOW ON Set the **Check** request's token to `{{apiKey2}}` and save it.

T19. Disabling a plan falls back to the default

Request	DELETE `{{baseUrl}}/v1/policies/{{premiumId}}`, Bearer `{{adminToken}}`

EXPECTED **204 No Content**. Send **Check** (with `{{apiKey2}}`): **200**, `"limit":5,"remaining":4`. Shop App is back on Free. GET `{{baseUrl}}/v1/policies/{{premiumId}}` shows `"enabled":false`: "delete" disables, it doesn't erase.

Proves: a disabled plan is skipped safely, and the caller drops to the default rather than to "unlimited".

T20. The admin API validates input

With Bearer `{{adminToken}}`:

Request	Body	Expected
`POST` `/v1/policies`	`{"name":"Bad","capacity":0,"window_seconds":60}`	**400** `capacity must be at least 1`
`POST` `/v1/clients`	`{"name":""}`	**400** `name is required`
`GET` `/v1/clients/01a1abee-0000-7000-8000-000000000000`	none	**404** `client not found`

(Put `{{baseUrl}}` in front of each path.)

T21. Unknown routes and wrong methods give clean JSON errors

Request	Expected
`GET` `{{baseUrl}}/v1/nothing`	**404** `no such route`
`GET` `{{baseUrl}}/v1/check` (check only accepts POST)	**405** `method not allowed; the Allow header lists the methods this path accepts`, and header `Allow: POST`

Proves: every error, even a routing miss, has the same JSON shape with a request ID, and no internal details.

T22. Your own request ID is passed back

Request	`GET` `{{baseUrl}}/healthz`
Headers tab	Add a row: Key `X-Request-ID`, Value `my-test-123`

`EXPECTED` **200**, and the response **Headers** tab shows `X-Request-Id: my-test-123`.

Proves: a calling system can trace one request through its own logs and RateGuard's.

T23. Usage analytics

Every decision is counted in memory, pushed to Redis every second and saved to Postgres every 10 seconds. So wait about **15 seconds** after your last Check, then:

Request	`GET` `{{baseUrl}}/v1/analytics/usage?top=3`, Bearer `{{adminToken}}`

`EXPECTED` **200 OK**. If you clicked exactly as this guide says, you get these numbers (an extra click changes them, which is fine):

```
{"from":"2026-11-04T14:00:00Z","to":"2026-11-05T14:00:00Z","granularity":"hour",
 "totals":       {"requests":21,"allowed":17,"rejected":4},
 "unidentified": {"requests":1,"allowed":1,"rejected":0},
 "clients":[{"client_id":"…","name":"Shop App","requests":20,"allowed":16,"rejected":4}],
 "series":[{"start":"…","requests":0,…}, … 24 hourly points …, {"start":"<this hour>","requests":21,…}]}
```

Check that: `requests = allowed + rejected`; **Shop App** is listed with a non-zero `rejected`; `unidentified` is the 1 keyless check from T12; and only the last hour in `series` is non-zero.

Only checks answered **200 or 429** are counted; peeks and errors (401, 403, 400) are not. Also try:

URL	Expected
`{{baseUrl}}/v1/analytics/usage?granularity=minute&top=5`	**200**, 1,440 one-minute points in `series`
`{{baseUrl}}/v1/analytics/usage?top=500`	**400** `top must be between 1 and 100`

Proves: usage is recorded without slowing decisions, and reports rank the most throttled applications first.

T24. Prometheus metrics

Request	`GET` `{{baseUrl}}/metrics`, No Auth (or open `http://127.0.0.1:8080/metrics` in a browser and press `Ctrl+F`)

`EXPECTED` **200**, a long plain-text page. Search for these lines (your numbers follow your clicks):

```
rateguard_decisions_total{outcome="allowed"} 17
rateguard_decisions_total{outcome="error"} 0
rateguard_decisions_total{outcome="no_policy"} 1
rateguard_decisions_total{outcome="rejected"} 4
rateguard_redis_breaker_state 0
rateguard_degraded_decisions_total{mode="local"} 0
rateguard_cache_lookups_total{cache="api_key",result="hit"} …
```

`no_policy 1` is T6. `breaker_state 0` means Redis is healthy. No line contains a client name, ID or key.

Proves: the service exposes what an operator needs (decisions, latency histogram, cache hit ratio, Redis health) without leaking caller identities.

T25. Swagger UI: the API documentation page

1. In the PowerShell terminal: `docker compose --profile docs up -d --wait swagger-ui`
2. Open `http://127.0.0.1:8081` in a browser.
3. Click any green or orange row (e.g. **POST /v1/check**) to expand it and read its request body, responses and examples.

`EXPECTED` A page titled **RateGuard** (OAS 3.1) with sections **decisions**, **health**, **clients**, **policies**, **analytics**. There is **no "Try it out"** button: that is deliberate (the page is read-only documentation; use Postman to send requests). The same document is served by RateGuard itself at `{{baseUrl}}/openapi.yaml`.

Part 3. Failure tests

What happens when a dependency dies. You stop a container with a command, test in Postman, then start it again. Run these after Part 2, in the VS Code terminal (PowerShell). The **Check** request should use `{{apiKey2}}`; Shop App is on Free (5 per 5 minutes) after T19.

F1. Redis goes down: decisions continue, still limited

#	Do this	Expected
1	`docker compose stop redis`	`Container rateguard-redis-1 Stopped`
2	GET `{{baseUrl}}/readyz`	**503** `{"status":"not ready","checks":{"postgres":"ok","redis":"unavailable"}}`
3	Send **Check** seven times	All answered, none hang: **200** five times (`remaining` 4, 3, 2, 1, 0), then **429** twice. The first one may take a moment longer. The limit still holds without Redis
4	Open `{{baseUrl}}/metrics`, search `degraded` and `breaker`	`rateguard_degraded_decisions_total{mode="local"} 7` (one per check in step 3) and `rateguard_redis_breaker_state 2` (2 = open)
5	`docker compose logs --since 5m rateguard`	JSON lines with `"level":"WARN"`, including `"msg":"redis circuit breaker opened: rate-limit decisions are degraded"`
6	`docker compose start redis`, wait **10 seconds**, send **Check**	**200** again, `"remaining":4` (see the note below)
7	Metrics again; logs again	`rateguard_redis_breaker_state 0`; an `"level":"INFO"` line `"redis circuit breaker closed: rate-limit decisions use redis again"`

Proves: a circuit breaker notices Redis is gone and switches to a local in-memory bucket. Decisions keep coming, and the limit **still applies**: RateGuard never turns into "allow everything". When Redis returns, it switches back by itself.

WHY REMAINING IS 4 AGAIN The development Redis is configured not to save anything to disk (bucket state is short-lived by design), so restarting it starts every bucket full. In production you would only lose the counts of an outage, never the plans.

F2. Postgres goes down: decisions continue from the cache

#	Do this	Expected
1	Send **Check** once, then run `docker compose stop postgres`	`Container rateguard-postgres-1 Stopped`
2	GET `{{baseUrl}}/readyz`	**503** `{"status":"not ready","checks":{"postgres":"unavailable","redis":"ok"}}`
3	GET `{{baseUrl}}/v1/policies` (admin token)	**500** `internal server error` after a few seconds. No SQL text, no host names: just the generic message and a request ID
4	Wait **15 seconds** (longer than the cache's 10-second lifetime), then send **Check** a few times	Normal answers, **200** or **429** with `"limit":5`, exactly as before the outage. The key and plan come from the last copy in memory
5	Open `{{baseUrl}}/metrics`, search `result="stale"`	`rateguard_cache_lookups_total{cache="api_key",result="stale"}` is above 0: those checks were answered from the saved copy
6	`docker compose start postgres`, wait 5 seconds, then `/readyz` and **Check**	`/readyz` **200** ready; Check answers normally. Send a few more checks a few seconds apart: the `stale` count stops rising

Proves: rate limiting doesn't depend on the database. RateGuard keeps deciding from the keys and plans it already knows, readiness reports the outage, and error messages never leak internals.

LIMITS OF THIS The saved copies are used for up to **15 minutes** of outage (ADR 0021). An application RateGuard has never seen before has nothing saved, so its checks fail with **500** until Postgres is back, and admin changes are impossible while the database is down.

F3. Secrets never reach the logs

1. In Postman, open the environment and copy the value of `apiKey2`.
2. In PowerShell (paste the key between the quotes):

```
docker compose logs rateguard | Select-String "PASTE-THE-FULL-KEY-HERE"
docker compose logs rateguard | Select-String "change-me-local-only-admin-token"
```

3. Now search for only the key's prefix: the part up to the **second** underscore after `rg_sho_` (e.g. `rg_sho_ql3shcqvkukewxho`).

EXPECTED The first two searches print **nothing**. Full keys and the admin token are never written to the logs; when a log line has to name a key, it uses only this prefix.

Part 4. Automated tests

The project has hundreds of automated tests. Run them from the VS Code terminal in the `RateGuard` folder. Docker Desktop must be running.

A1. Unit tests (quick, on Windows)

```
go test ./...
```

EXPECTED About a minute the first time. Every package line starts with `ok`; two say `[no test files]`, which is fine:

```
ok   github.com/Rimaz17/RateGuard/api                       1.383s
ok   github.com/Rimaz17/RateGuard/cmd/rateguard             6.828s
ok   github.com/Rimaz17/RateGuard/internal/analytics        …
ok   github.com/Rimaz17/RateGuard/internal/cache            …
ok   github.com/Rimaz17/RateGuard/internal/client           …
ok   github.com/Rimaz17/RateGuard/internal/config           …
ok   github.com/Rimaz17/RateGuard/internal/domain           …
ok   github.com/Rimaz17/RateGuard/internal/limiter          …
ok   github.com/Rimaz17/RateGuard/internal/policy           …
ok   github.com/Rimaz17/RateGuard/internal/storage/postgres …
ok   github.com/Rimaz17/RateGuard/internal/transport/http   …
?    github.com/Rimaz17/RateGuard/migrations                [no test files]
?    github.com/Rimaz17/RateGuard/test/load/socketbench     [no test files]
```

A failure prints `--- FAIL: TestName` and `FAIL` instead of `ok`.

IN VS CODE Open any `*_test.go` file (e.g. `internal/limiter/engine_test.go`). Above each `func Test…` there are small **run test | debug test** links: click one to run just that test. The **Testing** icon (flask) in the left bar lists every test with a green tick or red cross.

A2. Unit tests with the race detector

The project's rules require every test run to use `-race`, which catches two goroutines touching the same memory at once. It needs a C compiler that Windows doesn't have, so run it inside the official Go image (PowerShell):

```
docker run --rm -v "${PWD}:/src" -v rateguard-gomod:/go/pkg/mod -w /src golang:1.27.3 go test -race ./...
```

EXPECTED A few minutes the first time (it downloads the Go image and the modules), then faster. Same list of `ok` lines as A1, and no `WARNING: DATA RACE` anywhere in the output.

A3. The token bucket's coverage stays above 90%

```
docker run --rm -v "${PWD}:/src" -v rateguard-gomod:/go/pkg/mod -w /src golang:1.27.3 sh -c "go test -race -coverprofile=coverage.out ./... > /dev/null && make cover-engine"
```

EXPECTED Last line: `internal/limiter/engine.go: 100.0% of statements covered (minimum 90%)`

Proves: the algorithm at the heart of the service is fully tested. CI fails if this drops under 90%.

A4. Lint (code quality checks)

```
docker run --rm -v "${PWD}:/src" -w /src golangci/golangci-lint:v2.15.0 golangci-lint run
```

EXPECTED `0 issues.`

A5. Integration tests against real Postgres and Redis

These start their own throwaway Postgres and Redis containers (using *testcontainers*) and test the whole system, including several RateGuard instances sharing one limit and Redis being killed mid-test. One command, in PowerShell:

```
docker run --rm -v "${PWD}:/src" -v rateguard-gomod:/go/pkg/mod -v /var/run/docker.sock:/var/run/docker.sock
-e TESTCONTAINERS_HOST_OVERRIDE=host.docker.internal -w /src golang:1.27.3 go test -race -tags integration
./test/integration/...
```

EXPECTED About 4–5 minutes. One line at the end: `ok github.com/Rimaz17/RateGuard/test/integration 255.288s` (your time will differ). A failure shows `--- FAIL:` lines and `FAIL`.

NOTE The integration tests use their own containers, so they don't touch the stack you started in Part 1. Close heavy programs first: on a busy laptop a timing-sensitive test can occasionally be slow. Re-run before treating a single failure as a bug.

A6. The Go middleware example

`examples/go-middleware` is a ready-made piece of code a Go web app can use to call RateGuard. It has its own tests:

```
cd examples\go-middleware
go test ./...
```

EXPECTED `ok github.com/Rimaz17/RateGuard/examples/go-middleware` and `[no test files]` for `sampleapp`. One Unix-socket test is skipped on Windows by design.

End-to-end against your running RateGuard: create a new client in Postman (T5 with name `E2E App`; it lands on Free, limit 5), copy its `key` from the response, then:

```
$env:RATEGUARD_URL = "http://127.0.0.1:8080"
$env:RATEGUARD_API_KEY = "PASTE-THE-E2E-APP-KEY"
$env:RATEGUARD_LIMIT = "5"
go test -count=1 -run TestE2E -v .
cd ..\..
```

EXPECTED `--- PASS: TestE2E` then `ok`. Without the three variables the test prints `SKIP` instead, which is also fine.

A7. Three instances, one shared limit (the distributed demo)

Starts **three** RateGuard copies behind a load balancer and proves they share one limit, then stops Redis. Run in **Git Bash** (VS Code terminal → ˅ → Git Bash), in the `RateGuard` folder:

```
docker compose -f docker-compose.yml -f docker-compose.demo.yml up -d --build --wait
./scripts/demo-distributed.sh
```

EXPECTED The script prints what it does and ends without any `FAIL:` line. The key results:

```
== 1. 300 concurrent requests through the load balancer
allowed 100, refused 200
requests per instance:   (three addresses, about 100 each)
== 2. Stopping Redis, then 150 more requests
allowed 99, refused 51 (each instance's local bucket holds 100 / 3 = 33)
rateguard_redis_breaker_state per instance (2 = open):   2, 2, 2
== 3. Starting Redis again
rateguard_redis_breaker_state per instance (0 = closed): 0, 0, 0
== The demo passed: one shared limit across three instances, local enforcement
   while Redis was down, and recovery.
```

i.e. 300 requests sent at once for a client limited to 100: **exactly 100 allowed**, spread over all three instances; with Redis stopped every request is still answered from local buckets (a third of the limit each); after Redis returns the breakers close.

Afterwards, go back to the normal single-instance stack:

```
docker compose -f docker-compose.yml -f docker-compose.demo.yml down
docker compose up -d --wait
```

A8. Database migrations go down and back up

Command (PowerShell)	Expected last line
`docker compose run --rm migrate migrate version`	`{"level":"INFO","msg":"schema version","version":6,"dirty":false}`
`docker compose run --rm migrate migrate down 1`	`…"detail":"6/d create_usage_rollups …"` then `"version":5,"dirty":false`
`docker compose run --rm migrate migrate up`	`…"detail":"6/u create_usage_rollups …"` then `"version":6,"dirty":false`

CAREFUL `down 1` removes the usage table, so analytics from Part 2 are erased (plans, clients and keys stay).

A9. Load test (optional, about 25 minutes)

Measures speed (latency) and the maximum requests a second. Close other programs, then in **Git Bash**: `./scripts/loadtest.sh`. Reports are written to `test/load/results/`. Expect a median of about 1 ms per decision and a few thousand requests a second on a laptop; the README's **Performance** section has the published numbers. Results vary run to run (by 2–3× on a hot laptop). This test was not re-run for this guide.

Part 5. Finishing

5.1 Stop or reset everything

Goal	Command (PowerShell, in the RateGuard folder)
Stop, keep your data	`docker compose --profile docs down`
Stop and erase all local RateGuard data	`docker compose --profile docs down -v`
Start again	`docker compose up -d --wait` (add `--build` after changing code)
Watch the logs live	`docker compose logs -f rateguard` (stop with `Ctrl+C`)

5.2 Troubleshooting

Symptom	Cause and fix
error during connect … dockerDesktopLinuxEngine	Docker Desktop isn't running. Start it and wait for *Engine running*.
Postman: *Could not send request / ECONNREFUSED*	The stack isn't up. Run `docker compose ps`; start it with `docker compose up -d --wait`.
Postman URL shows `{{baseUrl}}` in red	No environment selected (top right) or a typo in the variable name.
Every check is **401 invalid API key**	`apiKey` is empty, or it was revoked in T18; use `{{apiKey2}}`. Make sure Auth Type is **Bearer Token**.
Check is **403** after T7	Free wasn't created with `"default": true`, or it was disabled. `GET /v1/policies` to look.
Numbers don't match the guide at all	The database wasn't empty when you started. Reset (5.1) and begin at T1.
Check is **400 request body is not valid JSON** on a good body	The Body drop-down is **Text**, not **JSON**, or there is a stray character.
`up` fails with *dependency postgres failed to start*	First start of a new database. Run `up` again.
port is already allocated	Another program (or another copy of the stack) uses the port. Change the port in `.env`, or stop the other one.
`go: command not found`	Go isn't installed or VS Code was opened before installing it. Install Go 1.27.3 and restart VS Code.
`curl` in PowerShell behaves oddly	Use `curl.exe`, or use Postman.

5.3 Checklist

✓	Test	✓	Test
☐	T1–T2 health and readiness	☐	T19 disabled plan falls back
☐	T3–T4 admin token required	☐	T20–T21 validation and JSON errors
☐	T5 register, key shown once	☐	T22 request ID echoed
☐	T6 no plan → 403	☐	T23 analytics
☐	T7–T8 five allowed, then 429	☐	T24 metrics
☐	T9 peek spends nothing	☐	T25 Swagger UI
☐	T10 refill	☐	F1 Redis down, still limited
☐	T11–T12 bad key 401, no key by IP	☐	F2 Postgres down
☐	T13 bad input 400	☐	F3 no secrets in logs
☐	T14 own plan	☐	A1–A4 unit, race, coverage, lint
☐	T15 endpoint rule	☐	A5 integration
☐	T16 live plan change	☐	A6 middleware
☐	T17 key never shown again	☐	A7 three-instance demo
☐	T18 key rotation	☐	A8 migrations